NewsRadar: Intermediate Project Update

A Geographical Visualization of News Around the Globe

Debasish Bordoloi
26M0776

October 17, 2026

GitHub Repository Link: https://www.github.com/debord555/cs699_project

1 Project Description

NewsRadar is a CS 699 (Software Lab) team project: a web application that scrapes news articles, extracts the places each article mentions, and plots them on an interactive map. It is built as a three-stage Python pipeline (scraping, location and topic mining, then a FastAPI backend with a React + Leaflet map) around a shared PostgreSQL + PostGIS database. The current version targets Indian news sources and geocodes places within India.

The main features of our application will be:

- News articles collected from multiple sources will be mapped to the locations they discuss, allowing users to explore what is happening across different regions of the world.
- News events from around the world will be represented as interactive points (blips) on a global map. Selecting a point will allow users to view the relevant news articles associated with that location.
- News websites will be continuously scraped to collect and update news data from multiple sources.
- Multiple articles reporting on the same real-world event will be identified and clustered together, preventing duplicate representation of the same event on the map.
- News will be categorized into different categories, including but not limited to Business, Sports, Military, Disasters, Crime, Environment, Technology, and Opportunities.
- Users will be able to filter news based on time and date ranges, allowing them to view recent or historical events.
- Users will be able to zoom into any geographical region and explore the news and events occurring in that area.
- The platform will provide links to the original news articles, allowing users to access the complete information from the respective news sources.
- Users will be able to search for news using text-based queries. The system will identify relevant news articles from across the world and display the corresponding events geographically on the map. For example, a search for "earthquake" would show relevant earthquake-related news from different locations around the world.

2 My Contributions

I worked on four areas, from the map frontend down to geocoding.

Area	What I did
Frontend	Created the `frontend/` app from the React + Vite template with ESLint, added `leaflet` and `react-leaflet`, and wrote `App.jsx`: a full-screen map with OpenStreetMap tiles, scroll-wheel zoom and a minimum zoom level. Article markers will be drawn on this base.
Infrastructure	Made `docker-compose.yml` work with Podman by using the fully qualified image name `docker.io/postgis/postgis:16-3.4`. Podman does not default to Docker Hub, so the short name failed; Docker users are unaffected.
Database	Designed the tables that store locations and link them to articles (Section 2.1).
Geocoding	Wrote `geocoding/geocode.py`, which turns place names into coordinates and caches them in the database (Section 2.2).

2.1 Location Schema

I added two tables and three indexes to `db/init/01_schema.sql`, so each place is stored once and linked to every article that mentions it.

Object	Purpose
`locations` table	One row per distinct place: `location_name`, `latitude`, `longitude`. CHECK constraints keep coordinates in valid ranges.
`article_location_map` table	Many-to-many link between articles and locations. `mention_order` records which place an article is mainly about. Deleting an article deletes its links.
Index on `lower(location_name)`	Case-insensitive name lookup, which lets the table act as the geocoding cache.
GiST index on `point(longitude, latitude)`	Bounding-box queries for the current map view, and nearest-first queries.
Index on `article_location_map (location_id)`	Fetching all articles about one place.

2.2 Geocoding Module

`geocoding/geocode.py` turns place names into coordinates using OpenStreetMap's Nominatim service through `geopy`, with the `locations` table acting as a cache.

`fetch_coordinates(name, hint)` searches within India only. The optional hint, such as a state, picks between places with the same name: for example, "Aurangabad" with the hint "Bihar" returns the town in Bihar, not the one in Maharashtra.

`get_or_create_location(conn, name, hint)` checks the `locations` table first and calls Nominatim only for names it has not seen. New places are inserted and their `id` is returned, ready

to link through `article_location_map`. Names Nominatim cannot find are stored with NULL coordinates so they are not looked up again, while network errors are raised instead of stored.

The module follows Nominatim's usage policy: at most 1 request per second, an identifying User-Agent, and cached results. It also runs from the command line:

```
python geocoding/geocode.py Guwahati Majuli --hint Assam
```

Testing. I checked the lookups against the live Nominatim service with Guwahati, Aurangabad with the Bihar hint, and a made-up name. The full insert into the database has not been run yet.